

SIMATS ENGINEERING ASSESSMENT TOOL 01

COURSE NAME: Ethical Hacking

COURSE CODE: ITA1410

COURSE OUTCOME COVERED

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.

QUESTIONS: 01

Total Marks:100

Annexure A SECURITY TESTING SIMULATION TASK

Code of Conduct:

I G. Nithya Sree (192424103) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Nithya Sree

Question 1: Web Server Reconnaissance and Vulnerability Analysis

Objective

To gather information about an authorized vulnerable web server, identify its services and technologies, discover hidden resources, and analyze security weaknesses.

Reconnaissance

Google and WHOIS are used to collect publicly available information such as domain details, subdomains, DNS information, and indexed pages.

Ping checks whether the server is reachable, while Traceroute shows the network path to the target. IPConfig provides network configuration details, and Netstat displays active connections and listening ports.

Nmap

Nmap identifies open ports, running services, and service versions. Unnecessary open ports increase the attack surface, while outdated services may contain known vulnerabilities.

WhatWeb

WhatWeb identifies web technologies such as the web server, programming language, CMS, and frameworks. This helps identify outdated or vulnerable components.

Curl/Wget

Curl or Wget can examine HTTP response headers. Headers may disclose server names, versions, technologies, and other information useful for attacker reconnaissance.

Nikto

Nikto scans the web server for common weaknesses such as outdated components, dangerous files, CGI issues, directory listing, and configuration problems.

Gobuster/Dirb

These tools discover hidden directories and files such as `/admin`, `/backup`, and `/config`. Exposed administrative, backup, or configuration files may reveal sensitive information.

Security Risks

Major risks include:

- Exposed credentials or configuration files
- Outdated/vulnerable software
- Unnecessary open ports
- Public administrative pages
- Directory listing
- Excessive server information disclosure

Recommendations

- Close unnecessary ports and services.
- Keep software updated.
- Remove unnecessary files and directories.
- Disable directory listing.
- Protect configuration and backup files.
- Restrict administrative pages.

The screenshot shows a Kali Linux virtual machine window titled "kali-linux-2025-virtualbox-amd64 [Running] - Oracle VM VirtualBox". The terminal window is titled "kali@kali: ~" and displays the following commands and output:

```

kali@kali: ~$ ping -c 4 192.168.1.20
PING 192.168.1.20 (192.168.1.20): 56(84) bytes of data:
 0: 192.168.1.20 ping statistics =
 4 packets transmitted, 0 received, 100% packet loss, time 3084ms

kali@kali: ~$ traceroute 192.168.1.20
traceroute to 192.168.1.20 (192.168.1.20), 30 hops max, 60 byte packets
 0: 192.168.1.2 (10.0.2.2) 23.000 ms 22.024 ms 1.303 ms
 1: *
 2: *
 3: *
 4: *
 5: *
 6: *
 7: *
 8: *
 9: *
10: *
11: *
12: *
13: *
14: *
15: *
16: *
17: *
18: *
19: *
20: *
21: *
22: *
23: *
24: *
25: *
26: *
27: *
28: *
29: *
30: *
31: *
32: *
33: *
34: *
35: *
36: *
37: *
38: *
39: *
40: *
41: *
42: *
43: *
44: *
45: *
46: *
47: *
48: *
49: *
50: *
51: *
52: *
53: *
54: *
55: *
56: *
57: *
58: *
59: *
60: *
61: *
62: *
63: *
64: *
65: *
66: *
67: *
68: *
69: *
70: *
71: *
72: *
73: *
74: *
75: *
76: *
77: *
78: *
79: *
80: *
81: *
82: *
83: *
84: *
85: *
86: *
87: *
88: *
89: *
90: *
91: *
92: *
93: *
94: *
95: *
96: *
97: *
98: *
99: *
100: *
101: *
102: *
103: *
104: *
105: *
106: *
107: *
108: *
109: *
110: *
111: *
112: *
113: *
114: *
115: *
116: *
117: *
118: *
119: *
120: *
121: *
122: *
123: *
124: *
125: *
126: *
127: *
128: *
129: *
130: *
131: *
132: *
133: *
134: *
135: *
136: *
137: *
138: *
139: *
140: *
141: *
142: *
143: *
144: *
145: *
146: *
147: *
148: *
149: *
150: *
151: *
152: *
153: *
154: *
155: *
156: *
157: *
158: *
159: *
160: *
161: *
162: *
163: *
164: *
165: *
166: *
167: *
168: *
169: *
170: *
171: *
172: *
173: *
174: *
175: *
176: *
177: *
178: *
179: *
180: *
181: *
182: *
183: *
184: *
185: *
186: *
187: *
188: *
189: *
190: *
191: *
192: *
193: *
194: *
195: *
196: *
197: *
198: *
199: *
200: *
201: *
202: *
203: *
204: *
205: *
206: *
207: *
208: *
209: *
210: *
211: *
212: *
213: *
214: *
215: *
216: *
217: *
218: *
219: *
220: *
221: *
222: *
223: *
224: *
225: *
226: *
227: *
228: *
229: *
230: *
231: *
232: *
233: *
234: *
235: *
236: *
237: *
238: *
239: *
240: *
241: *
242: *
243: *
244: *
245: *
246: *
247: *
248: *
249: *
250: *
251: *
252: *
253: *
254: *
255: *
256: *
257: *
258: *
259: *
260: *
261: *
262: *
263: *
264: *
265: *
266: *
267: *
268: *
269: *
270: *
271: *
272: *
273: *
274: *
275: *
276: *
277: *
278: *
279: *
280: *
281: *
282: *
283: *
284: *
285: *
286: *
287: *
288: *
289: *
290: *
291: *
292: *
293: *
294: *
295: *
296: *
297: *
298: *
299: *
300: *
301: *
302: *
303: *
304: *
305: *
306: *
307: *
308: *
309: *
310: *
311: *
312: *
313: *
314: *
315: *
316: *
317: *
318: *
319: *
320: *
321: *
322: *
323: *
324: *
325: *
326: *
327: *
328: *
329: *
330: *
331: *
332: *
333: *
334: *
335: *
336: *
337: *
338: *
339: *
340: *
341: *
342: *
343: *
344: *
345: *
346: *
347: *
348: *
349: *
350: *
351: *
352: *
353: *
354: *
355: *
356: *
357: *
358: *
359: *
360: *
361: *
362: *
363: *
364: *
365: *
366: *
367: *
368: *
369: *
370: *
371: *
372: *
373: *
374: *
375: *
376: *
377: *
378: *
379: *
380: *
381: *
382: *
383: *
384: *
385: *
386: *
387: *
388: *
389: *
390: *
391: *
392: *
393: *
394: *
395: *
396: *
397: *
398: *
399: *
400: *
401: *
402: *
403: *
404: *
405: *
406: *
407: *
408: *
409: *
410: *
411: *
412: *
413: *
414: *
415: *
416: *
417: *
418: *
419: *
420: *
421: *
422: *
423: *
424: *
425: *
426: *
427: *
428: *
429: *
430: *
431: *
432: *
433: *
434: *
435: *
436: *
437: *
438: *
439: *
440: *
441: *
442: *
443: *
444: *
445: *
446: *
447: *
448: *
449: *
450: *
451: *
452: *
453: *
454: *
455: *
456: *
457: *
458: *
459: *
460: *
461: *
462: *
463: *
464: *
465: *
466: *
467: *
468: *
469: *
470: *
471: *
472: *
473: *
474: *
475: *
476: *
477: *
478: *
479: *
480: *
481: *
482: *
483: *
484: *
485: *
486: *
487: *
488: *
489: *
490: *
491: *
492: *
493: *
494: *
495: *
496: *
497: *
498: *
499: *
500: *
501: *
502: *
503: *
504: *
505: *
506: *
507: *
508: *
509: *
510: *
511: *
512: *
513: *
514: *
515: *
516: *
517: *
518: *
519: *
520: *
521: *
522: *
523: *
524: *
525: *
526: *
527: *
528: *
529: *
530: *
531: *
532: *
533: *
534: *
535: *
536: *
537: *
538: *
539: *
540: *
541: *
542: *
543: *
544: *
545: *
546: *
547: *
548: *
549: *
550: *
551: *
552: *
553: *
554: *
555: *
556: *
557: *
558: *
559: *
560: *
561: *
562: *
563: *
564: *
565: *
566: *
567: *
568: *
569: *
570: *
571: *
572: *
573: *
574: *
575: *
576: *
577: *
578: *
579: *
580: *
581: *
582: *
583: *
584: *
585: *
586: *
587: *
588: *
589: *
590: *
591: *
592: *
593: *
594: *
595: *
596: *
597: *
598: *
599: *
600: *
601: *
602: *
603: *
604: *
605: *
606: *
607: *
608: *
609: *
610: *
611: *
612: *
613: *
614: *
615: *
616: *
617: *
618: *
619: *
620: *
621: *
622: *
623: *
624: *
625: *
626: *
627: *
628: *
629: *
630: *
631: *
632: *
633: *
634: *
635: *
636: *
637: *
638: *
639: *
640: *
641: *
642: *
643: *
644: *
645: *
646: *
647: *
648: *
```

Question 2: Web Application Vulnerability Assessment

Objective

To assess an authorized vulnerable web application, identify its technologies and attack surface, test common web vulnerabilities, and recommend suitable security controls.

Reconnaissance

Nmap is used to identify open ports, services, and versions: `nmap -sV 192.168.1.20`

This helps identify the web server and other exposed services.

Web-Server Scanning

Nikto is used to identify common web-server weaknesses: `nikto -h http://192.168.1.20`

It can detect outdated components, dangerous files, directory listing, and security misconfigurations.

HTTP Analysis

Curl is used to examine HTTP responses and headers: `curl -i http://192.168.1.20`

This helps identify server information, cookies, status codes, and other response details.

Burp Suite

Burp Suite is used to intercept and analyze HTTP requests and responses. It helps examine:

- Input fields and parameters
- Cookies
- Sessions
- Authentication requests
- GET and POST requests
- Server responses

The captured requests can be reviewed to identify possible SQL injection, XSS, authentication, and session-management weaknesses.

OWASP ZAP

OWASP ZAP is used to scan and analyze the web application for common vulnerabilities. It can identify issues such as XSS, insecure configurations, and other web-security weaknesses.

Browser Developer Tools

Developer Tools (F12) are used to inspect:

- HTTP requests
- Cookies
- Session information
- JavaScript
- Responses
- Network activity

Vulnerability Analysis

SQL Injection:

Input fields and parameters are checked to determine whether user input is improperly handled by the database. SQL injection may allow unauthorized database access.

Cross-Site Scripting (XSS):

Input and output are checked to determine whether user-supplied scripts can be executed in a user's browser. XSS may lead to session theft or malicious actions.

Insecure Authentication:

Login mechanisms are examined for weak authentication controls, poor password handling, or improper access restrictions.

Session Management:

Cookies and session tokens are analyzed for weaknesses such as insecure attributes, predictable tokens, or improper session expiration.

Directory Traversal:

Parameters that access files are tested to determine whether unauthorized files outside the intended directory can be accessed.

Severity Classification

Severity	Example	Impact
Critical	Confirmed SQL injection allowing major unauthorized access	Severe compromise

High	Stored XSS or authentication bypass	Account/data compromise
Medium	Reflected XSS or session weakness	User/session compromise
Low	Information disclosure	Helps attacker reconnaissance

Recommendations

1. Validate and sanitize all user input.
2. Use parameterized SQL queries to prevent SQL injection.
3. Encode output to prevent XSS.
4. Use strong authentication and authorization controls.
5. Protect cookies using **Secure**, **HttpOnly**, and appropriate **SameSite** settings.
6. Implement proper session expiration and regeneration.
7. Restrict file access to prevent directory traversal.
8. Remove unnecessary services and exposed files.
9. Keep web-server and application components updated.
10. Perform regular security testing.

Conclusion

Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser Developer Tools provide different views of the web application. Combining these tools helps identify vulnerabilities in input handling, authentication, sessions, file access, and server configuration. The identified vulnerabilities should be verified in the authorized laboratory environment and prioritized according to their severity and potential impact.

```

kali@kali:~$ nmap -sV 192.168.1.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-18 12:36 EDT
Nmap scan report for 192.168.1.20
Host is up (0.0017s latency).
All 1000 scanned ports on 192.168.1.20 are in ignored states.
Not shown: 1000 filtered tcp ports (no response)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 12.33 seconds

kali@kali:~$ curl -X GET http://192.168.1.20
curl: (22) Failed to connect to 192.168.1.20 port 80 after 21005 ms: Could not connect to server

kali@kali:~$ nmap -sV 192.168.1.20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-18 12:56 EDT
Nmap scan report for 192.168.1.20
Host is up (0.0017s latency).
All 1000 scanned ports on 192.168.1.20 are in ignored states.
Not shown: 1000 filtered tcp ports (no response)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 4.55 seconds

kali@kali:~$ nikto -u http://192.168.1.20
- Nikto v2.5.0

+ 0 Host(s) tested

kali@kali:~$ curl -X GET http://192.168.1.20
curl: (22) Failed to connect to 192.168.1.20 port 80 after 21017 ms: Could not connect to server

kali@kali:~$ burpsuite
Your Java appears to be version 21.0.9 from Debian
Burp has not been fully installed on this platform and you may experience problems.

```

Question 3: Security Header and HTTPS Configuration Assessment

Objective

The objective is to examine the authorized web application's HTTP security headers and HTTPS/TLS configuration, identify weaknesses, determine their security impact, and recommend appropriate security controls.

Tools Used

- Curl
- Nmap SSL scripts

1. HTTP Security Headers

Curl is used to examine the HTTP response headers:

```
curl -I http://TARGET-IP
```

Important headers to check are:

- Content-Security-Policy (CSP): Helps prevent XSS and other code-injection attacks.
- Strict-Transport-Security (HSTS): Forces browsers to use HTTPS.
- X-Frame-Options: Helps prevent clickjacking.
- X-Content-Type-Options: Prevents browsers from incorrectly interpreting content types.

If these headers are missing or incorrectly configured, the application may have increased exposure to browser-based attacks.

2. HTTP to HTTPS Redirect

Use:

```
curl -IL http://TARGET-IP
```

A secure configuration should redirect HTTP requests to HTTPS, normally using a **301** or **302** response with a **Location: https://...** header.

If HTTP remains accessible without redirection, unencrypted communication may be possible.

3. HTTPS Header Check

```
curl -I https://TARGET-IP
```

This confirms which security headers are provided over HTTPS.

4. TLS/Cipher Assessment

Nmap can examine the supported TLS protocols and cipher suites:

```
nmap -p 443 --script ssl-enum-ciphers TARGET-IP
```

This helps identify:

- Supported TLS versions
- Weak protocols
- Weak cipher suites
- Strongly configured encryption

Old or weak protocols and ciphers should be disabled.

5. Certificate Assessment

Use:

```
nmap -p 443 --script ssl-cert TARGET-IP
```

The certificate should be checked for:

- Correct domain name
- Validity period
- Trusted issuer
- Expired certificate
- Incorrect certificate configuration

Findings and Severity

Finding	Severity	Impact
Missing CSP	Medium	Increased XSS risk
Missing HSTS	Medium	Allows potential downgrade/HTTP exposure
Missing X-Frame-Options	Medium	Increased clickjacking risk
Missing X-Content-Type-Options	Low	MIME-sniffing risk

HTTP not redirected to HTTPS	High	Sensitive traffic may be unencrypted
Weak TLS protocol	High	Reduced encryption security
Weak cipher suite	Medium/High	Weaker protection of communications
Expired/invalid certificate	High	Trust and secure communication problems

Recommendations

1. Enable a suitable Content-Security-Policy.
2. Enable HSTS after confirming HTTPS is correctly deployed.
3. Configure `X-Frame-Options` or an appropriate CSP `frame-ancestors` policy.
4. Add `X-Content-Type-Options: nosniff`.
5. Redirect all HTTP requests to HTTPS.
6. Disable obsolete TLS protocols and weak cipher suites.
7. Use current secure TLS configurations.
8. Use a valid certificate issued by a trusted CA.
9. Renew certificates before expiration.
10. Periodically reassess HTTPS and security-header configurations.

Conclusion

Curl and Nmap SSL scripts provide information about the application's HTTP headers, HTTPS redirection, TLS protocols, cipher suites, and certificate configuration. Missing security headers and weak TLS settings can increase the application's attack surface and reduce protection against web-based attacks. Proper HTTPS configuration, secure TLS settings, valid certificates, and appropriate security headers significantly improve the application's security.

[illegible]

Question 4: Broken Access-Control Assessment

Objective

The objective is to test whether the authorized student portal correctly enforces access control between **student and administrator accounts** and between different student records.

Tool Used

OWASP ZAP

1. Testing Procedure

First, log in using the provided **student test account** and access the student's own record.

In OWASP ZAP, capture the request generated by the application.

Look for identifiers such as:

- Student ID
- User ID
- Record ID
- Object ID
- URL parameters
- Form parameters
- API parameters
- Session cookies

Example:

GET /student/profile?id=101

The value **101** is an object identifier.

Use only the test identifiers provided by your faculty.

2. Test Another Student's Record

Using the authorized test identifier provided by the faculty, modify the request in ZAP.

For example:

GET /student/profile?id=102

Send the modified request and compare the response with the original request.

Secure Result

The server should reject the request, for example:

403 Forbidden

or redirect the user to an appropriate error/login page.

Vulnerable Result

If the normal student account receives another student's private record with a successful response such as:

200 OK

then the application may have an **Insecure Direct Object Reference (IDOR)** / broken object-level authorization vulnerability.

3. Test Administrator Resources

Log in using the normal student account and identify an administrator-only request using the provided test environment.

For example:

GET /admin/dashboard

Send the request while authenticated as the student.

Secure Result

The server should reject the request:

403 Forbidden

or another appropriate authorization response.

Vulnerable Result

If the student account can successfully access administrator-only information or functionality, this indicates a **broken function-level authorization** weakness.

4. Compare Server Responses

Test	Expected Secure Response	Vulnerable Response
Student accesses own record	200 OK	200 OK
Student accesses another student's record	403/denied	200 OK with data
Student accesses admin resource	403/denied	200 OK
Unauthenticated access	Login/401	Sensitive data accessible

The response should be checked for both **HTTP status codes and returned content**. A **200 OK** response does not always mean the request was authorized, so the actual response body must also be examined.

5. Authentication vs Authorization

Authentication answers:

"Who are you?"

For example, successfully logging in with a student username and password.

Authorization answers:

"What are you allowed to access?"

For example, a student may be authenticated but should only be authorized to view their own record.

Therefore:

Authentication failure: The user cannot prove their identity.

Authorization failure: The user is authenticated but is allowed to access something they should not.

6. Security Impact

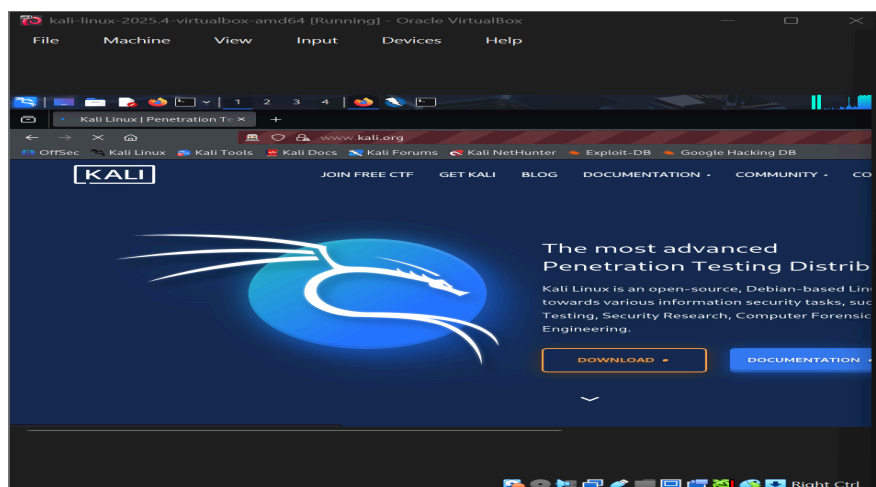
A broken access-control vulnerability could allow:

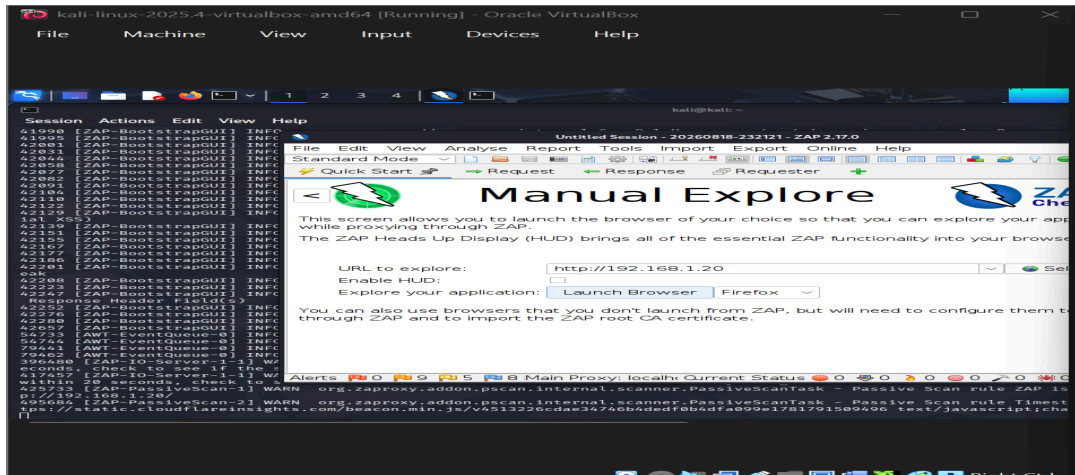
- One student to view another student's information.
- Students to access administrator resources.
- Unauthorized modification of records.
- Exposure of personal or academic information.
- Privilege escalation.

The severity depends on what information or functionality can be accessed.

Conclusion

OWASP ZAP can capture and modify requests to verify whether access controls are properly enforced. A student should only be able to access their own authorized resources, while administrator functions must remain restricted to administrator accounts. The most serious finding would be a confirmed ability for a student to access another student's confidential information or administrator-only functionality. Proper server-side authorization, role validation, object ownership checks, and least-privilege controls are required to prevent broken access-control vulnerabilities.





Question 5: Automated Scanning and Manual Validation

Objective

The objective is to identify vulnerabilities in an authorized web application using **Nikto** and **OWASP ZAP/Wapiti**, compare their results, manually validate important findings, and recommend suitable remediation.

1. Nikto Scan

Nikto is used to identify web-server misconfigurations, outdated components, exposed files, and information-disclosure issues.

Example command:

```
nikto -h http://TARGET-IP
```

The results should be recorded with screenshots.

Possible Findings

- Outdated web-server software
- Missing security headers
- Exposed files
- Directory listing
- Default files
- Server information disclosure

2. OWASP ZAP Automated Scan

Start ZAP:

zaproxy

Enter the authorized laboratory application's URL in ZAP and perform an automated scan.

ZAP can identify:

- Cross-Site Scripting
- Security-header problems
- Authentication/session issues
- Injection-related issues
- Information disclosure
- Other web-application weaknesses

3. Comparison of Results

Category	Nikto	OWASP ZAP
Misconfiguration	Web-server issues	Web/application configuration
Information disclosure	Server/files/headers	Headers/application responses
Authentication	Limited	More suitable for application testing
Injection	Limited	Better suited for web-application testing
Exposed resources	Strong	Can identify application resources

Nikto mainly focuses on the **web server**, while ZAP provides broader **web-application security testing**.

4. Manual Validation

At least three important findings should be manually checked.

Finding 1 – Security Header Missing

If a scanner reports a missing security header, check the response:

```
curl -I http://TARGET-IP
```

Check for headers such as:

Content-Security-Policy

Strict-Transport-Security

X-Frame-Options

X-Content-Type-Options

Classification: Configuration weakness.

Severity: Low/Medium depending on the application.

Validation: Confirm whether the header is actually absent.

Remediation: Configure the appropriate security headers on the web server/application.

Finding 2 – Exposed File or Directory

If Nikto/ZAP reports an exposed resource, access only the identified test resource in the authorized lab.

Example:

```
curl -I http://TARGET-IP/identified-resource
```

If the resource is accessible when it should not be:

Classification: Information disclosure/configuration weakness.

Severity: Medium/High depending on the information exposed.

Remediation: Remove unnecessary files and restrict access to sensitive resources.

Finding 3 – XSS or Injection Finding

If ZAP reports a possible XSS/injection issue, manually inspect the affected parameter using the ZAP request editor and the application's test input.

Do not treat an automated alert as automatically confirmed.

Confirmed: The application demonstrably processes the test input in an unsafe way.

False Positive: The input is safely encoded/handled and the reported condition cannot be reproduced.

Further Investigation: The scanner reports a possible weakness but additional controlled testing is required.

Remediation: Validate input, encode output, and use secure application-development practices such as parameterized queries for database operations

5. Finding Classification

Finding	Category	Possible Severity
Missing security headers	Misconfiguration	Low/Medium
Server version disclosure	Information disclosure	Low
Exposed sensitive file	Information disclosure	Medium/High
Weak authentication	Authentication	High
XSS	Injection	Medium/High

SQL injection	Injection	High/Critical
---------------	-----------	---------------

Outdated component	Misconfiguration	Medium/High
--------------------	------------------	-------------

The final severity should be based on the **actual evidence and impact**, not simply the scanner's rating.

6. Confirmed, False Positive, or Further Investigation

Each important scanner finding should be classified as:

Confirmed

The finding was reproduced during manual validation and its security impact was verified.

False Positive

The scanner reported an issue, but manual testing showed that the application is properly protected.

Requires Further Investigation

The finding appears possible, but more controlled testing is required before confirming it.

7. Remediation Measures

Misconfiguration

- Harden the web server.
- Remove unnecessary services and files.
- Disable directory listing.
- Apply secure configuration settings.

Information Disclosure

- Remove unnecessary server-version information.
- Protect sensitive files.
- Configure secure HTTP headers.

Authentication

- Use strong authentication.
- Apply proper session management.
- Implement MFA where appropriate.
- Enforce server-side authorization.

Injection

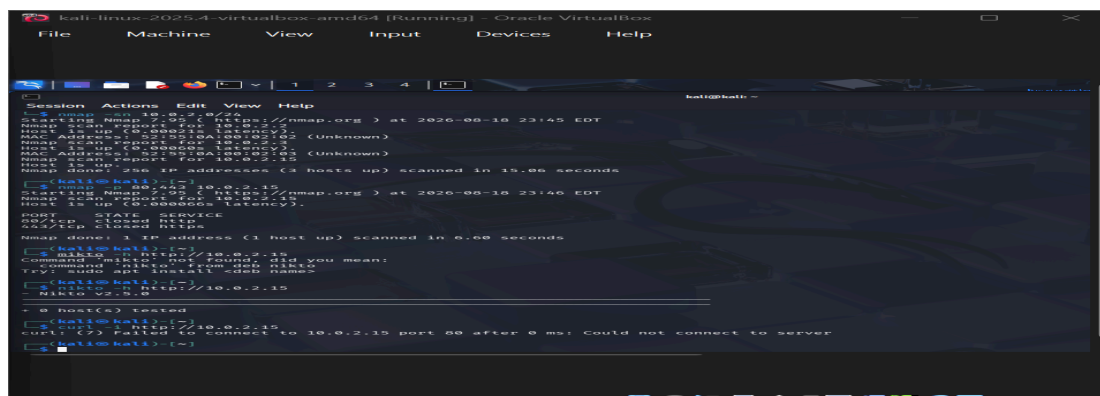
- Validate input.
- Encode output.
- Use parameterized SQL queries.
- Avoid unsafe dynamic command/query construction.

Outdated Components

- Update the web server and application components.
- Apply security patches regularly.
- Remove unsupported software.

Conclusion

Nikto and OWASP ZAP provide complementary information. Nikto primarily identifies web-server misconfigurations, outdated components, and exposed resources, while OWASP ZAP provides broader web-application vulnerability detection and request-level analysis. Automated scanner results should not automatically be considered confirmed vulnerabilities. At least three significant findings should be manually validated and classified as confirmed, false positive, or requiring further investigation. The most serious confirmed vulnerabilities should be remediated first, particularly injection vulnerabilities, authentication weaknesses, and exposure of sensitive information. Regular automated scanning combined with manual validation provides a more reliable assessment of the application's security posture.



```
kali@kali: ~$ nmap -sS -p 80 10.0.2.15
Nmap scan report for 10.0.2.15
Host is up (0.0000000 latency).
PORT: 80/tcp
Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds

kali@kali: ~$ curl -I http://10.0.2.15
HTTP/1.1 200 OK
Content-Type: text/html
Server: Apache/2.4.18 (Ubuntu)
Date: Mon, 10 Sep 2018 23:45:15 GMT
Connection: close
curl: (22) Failed to connect to 10.0.2.15 port 80 after 0 ms: Could not connect to server

kali@kali: ~$
```